

SQL Server 备份与还原实验报告 (Lecture 5)

【23336128】

【梁力航】

实验信息

- 实验名称：备份与还原策略实践 (lc5)
- 实验日期：2025-11-20
- 数据库：School_Data （运行在 Docker 容器 sqlserver2019 中）
- 实验目标：
 - i. 生成完整备份、差异备份与事务日志备份
 - ii. 在执行插入/删除/更新后，使用备份链还原数据库并对比状态

环境与工具

1. 运行环境

- 操作系统：macOS + Docker Desktop
- 数据库版本：SQL Server 2019 容器（容器名：sqlserver2019）
- 客户端工具：sqlcmd（路径：/opt/mssql-tools18/bin/sqlcmd）
- 数据库文件挂载：~/Downloads/Database/School_Data -> /var/opt/mssql/data
- 备份目录：~/Downloads/Database/School_Data/backup -> /var/opt/mssql/data/backup

2. 备份脚本 scripts/sql_backup.sh

该脚本用于执行不同类型的数据库备份操作，支持完整备份、差异备份和事务日志备份三种模式。

核心逻辑如下：

```
SQLCMD=${SQLCMD_PATH:-/opt/mssql-tools18/bin/sqlcmd}
SQLCMD_OPTS="-S localhost -U sa -P "$SA_PASSWORD" -C"
case "$action" in
    full) SQL="BACKUP DATABASE [$DB_NAME] TO DISK = N'...full...bak' WITH INIT, STATS=5;" ;;
    diff) SQL="BACKUP DATABASE [$DB_NAME] TO DISK = N'...diff...bak' WITH DIFFERENTIAL, INIT, STATS=5;" ;;
    log)  SQL="BACKUP LOG [$DB_NAME] TO DISK = N'...log...trn' WITH INIT, STATS=5;" ;;
esac
docker exec "$CONTAINER_NAME" bash -c "$SQLCMD $SQLCMD_OPTS -Q \"$SQL\""
```

3. 还原脚本 scripts/sql_restore.sh

该脚本实现完整的数据库还原流程，包括将数据库切换为单用户模式、按顺序应用完整备份、差异备份和事务日志备份，最后再切换回多用户模式。

主要功能代码：

```
resolve_host_path() { ... } # 校验备份文件并映射至容器路径
run_restore() {
    docker exec "$CONTAINER_NAME" bash -c "$SQLCMD $SQLCMD_OPTS -Q \"sql\""
}
run_restore "RESTORE DATABASE ... FROM <full> WITH REPLACE, NORECOVERY"
run_restore "RESTORE DATABASE ... FROM <diff> WITH NORECOVERY"
run_restore "RESTORE LOG ... FROM <log> WITH RECOVERY"
```

实验步骤与执行过程

步骤 1：创建完整备份

```
$ ./scripts/sql_backup.sh full
执行 full 备份，输出文件：.../School_Data_full_20251120_220534.bak
...
BACKUP DATABASE successfully processed 4634 pages ...
```

步骤 2：插入测试数据（模拟业务变更）

```
$ docker exec ... -d School_Data -Q "
    INSERT INTO Worker(Number,Name,Sex,Sage,Department)
    VALUES('99999','BkTest','M',20,'BackupLab');
    SELECT Number,Name,Department FROM Worker WHERE Number='99999';"
(1 rows affected)
99999 BkTest BackupLab
```

步骤 3：创建差异备份

```
$ ./scripts/sql_backup.sh diff
执行 diff 备份 ... School_Data_diff_20251120_220740.bak
BACKUP DATABASE WITH DIFFERENTIAL successfully processed 106 pages ...
```

步骤 4：更新数据并执行事务日志备份

```
$ docker exec ... -Q "
    UPDATE Worker SET Department='AfterDiff' WHERE Number='99999';
    SELECT Number,Department FROM Worker WHERE Number='99999';"
99999 AfterDiff

$ ./scripts/sql_backup.sh log
执行 log 备份 ... School_Data_log_20251120_220757.trn
BACKUP LOG successfully processed 796 pages ...
```

步骤 5：删除数据（模拟误操作场景）

```
$ docker exec ... -Q "
    DELETE FROM Worker WHERE Number='99999';
    SELECT Number,Department FROM Worker WHERE Number='99999';"
(0 rows affected)
```

步骤 6：按照备份链进行数据库还原

```
$ ./scripts/sql_restore.sh \
    --full School_Data/backup/School_Data_full_20251120_220534.bak \
    --diff School_Data/backup/School_Data_diff_20251120_220740.bak \
    --log School_Data/backup/School_Data_log_20251120_220757.trn
将数据库切换到 SINGLE_USER 模式...
RESTORE DATABASE successfully processed ... (full)
RESTORE DATABASE successfully processed ... (diff)
RESTURE LOG successfully processed ... (log)
切回 MULTI_USER 模式
还原完成!
```

步骤 7：验证还原结果

```
$ docker exec ... -Q "SELECT Number,Department FROM Worker WHERE Number='99999';"
99999 AfterDiff
```

记录被成功恢复，并保留了事务日志备份中的最新部门值 AfterDiff 。

结果分析

阶段	Worker 表中 Number=99999 的状态
插入后 + 差异备份前	Department = BackupLab
更新后 + 日志备份前	Department = AfterDiff

阶段	Worker 表中 Number=99999 的状态
删除后（模拟故障）	不存在
还原完成后	Department = AfterDiff （与日志备份一致）

- **备份策略验证：** sql_backup.sh 成功生成了完整、差异、日志三类备份文件，满足实验要求 ①。
- **还原验证：** 通过 sql_restore.sh 按照备份链正确恢复，成功找回被删除的数据，满足实验要求 ②。
- **脚本优势：** 脚本封装了容器启动、SSL 信任、模式切换、路径映射等复杂操作，确保在 macOS + Docker 环境下可以稳定重复执行。

实验总结

技术要点总结

1. **备份链的重要性：** 完整备份、差异备份和事务日志备份构成了完整的恢复机制，只有按顺序保留这三种备份，才能在出现误操作后精确地恢复到指定时间点。
2. **脚本化管理的优势：** 通过脚本自动完成数据库模式切换、TLS 参数配置等复杂操作，大大降低了日常备份和还原的操作复杂度和出错概率。

实践体会

1. 数据库备份不仅仅是简单的数据复制，而是一个包含多种备份类型、严格顺序要求的技术体系。
2. 通过本次实验，深入理解了 SQL Server 不同类型备份的特点和应用场景：
 - 完整备份：作为恢复的基础，包含数据库所有数据
 - 差异备份：只包含自上次完整备份以来更改的数据，节省存储空间和备份时间
 - 事务日志备份：记录所有事务操作，支持精确时间点恢复
3. 自动化脚本对于数据库运维工作具有重要意义，不仅能提高工作效率，还能减少人为错误的发生。

改进建议

1. 可以考虑增加备份文件的压缩和加密功能，提升备份的安全性和存储效率。
2. 建议添加更完善的错误处理机制，使脚本在遇到异常情况时能够提供更有价值的错误信息。
3. 可以扩展脚本功能，支持定时自动备份，进一步提升数据库管理的自动化水平。